

Introduction to GPGPU (General-Purpose Graphics Processing Units) and OpenCL

Yixing Huang, Siyuan Mei, Mareike Thies, Prof. Andreas Maier



FRIEDRICH-ALEXANDER
UNIVERSITÄT
ERLANGEN-NÜRNBERG

TECHNISCHE FAKULTÄT

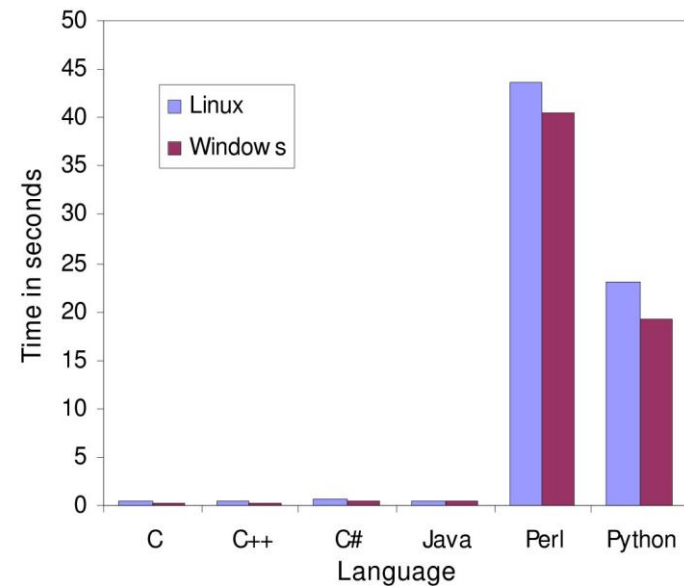
Python

- Python: many advantages, but very slow

Slowest things on earth:



Source: medium.com



Source: [\[1\]](#)

[1] Fourment M, Gillings MR. A comparison of common programming languages used in bioinformatics. BMC bioinformatics. 2008 Dec;9(1):1-9.

GPU Acceleration

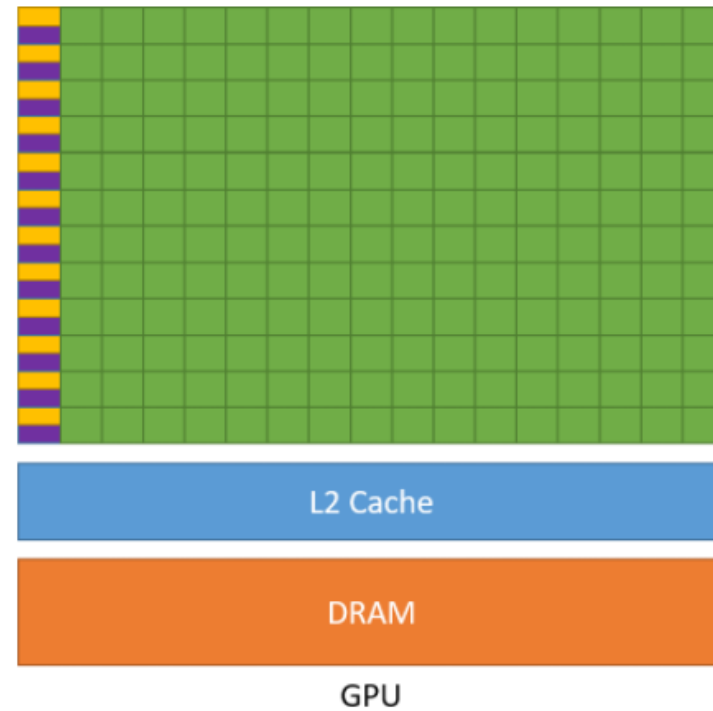
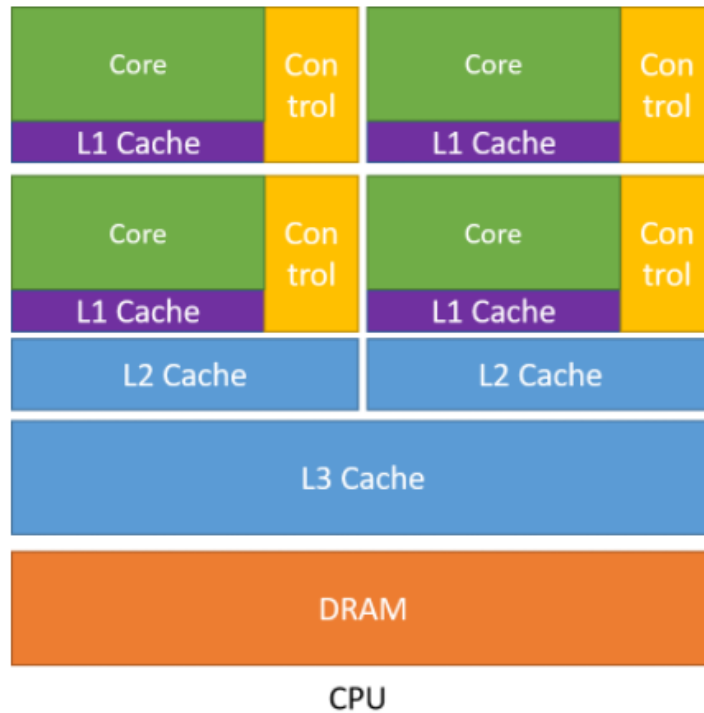
- GPU (Graphics Processing Unit)
 - Processing of the traditional graphic pipeline like geometry and textures
 - Parallelism
 - OpenGL (Prof. Marc Stamminger, Interactive Computer Graphics, Computergraphik, FAU video portal)
- GPGPU
 - Programmable processors replaced fixed function blocks
 - Bidirectional data flow between CPU and GPU
 - Programming languages: [CUDA](#), [OpenCL](#), [Metal](#)



Nvidia GTX 2080

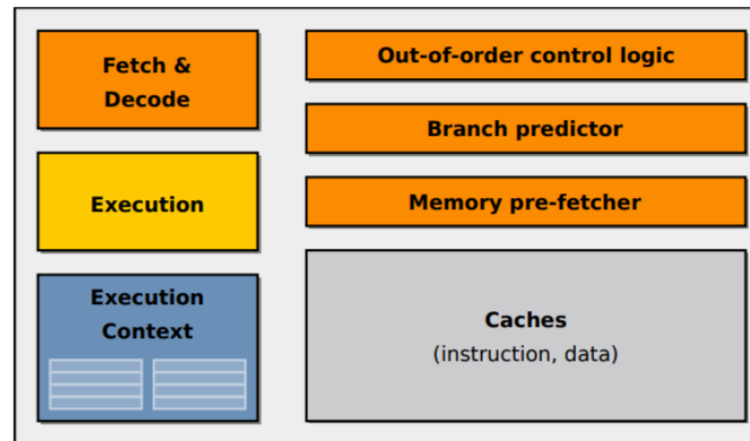
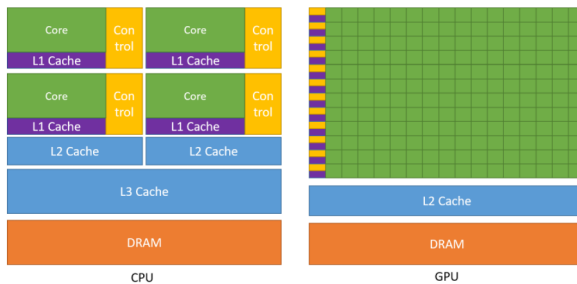
GPU vs. CPU

- GPU has much more processing cores than CPU

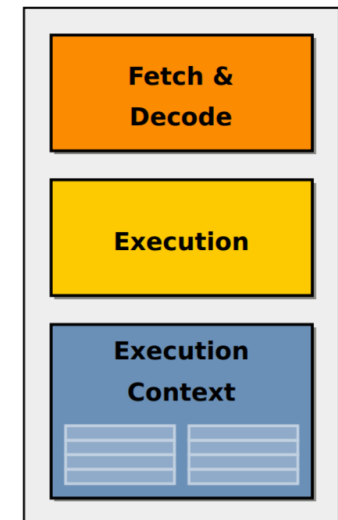


GPU vs. CPU

- GPU has much more processing cores than CPU
- GPU's processing cores are simpler than CPU's



A basic CPU core



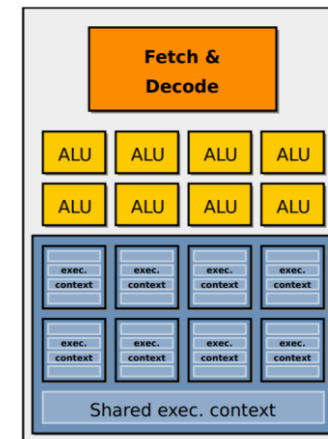
A simple processing core

Single Instruction Multiple Data (SIMD)

- Process multiple (e.g., 16) instruction streams in parallel
- Share instructions between streams (e.g., 16 -> 2)
- Instead of one decode & fetch unit per instruction stream, the decode & fetch unit is reused for multiple instruction streams
- Example: apply the same instruction to $1280 * 1024$ pixels



Example: GPU with 16 cores

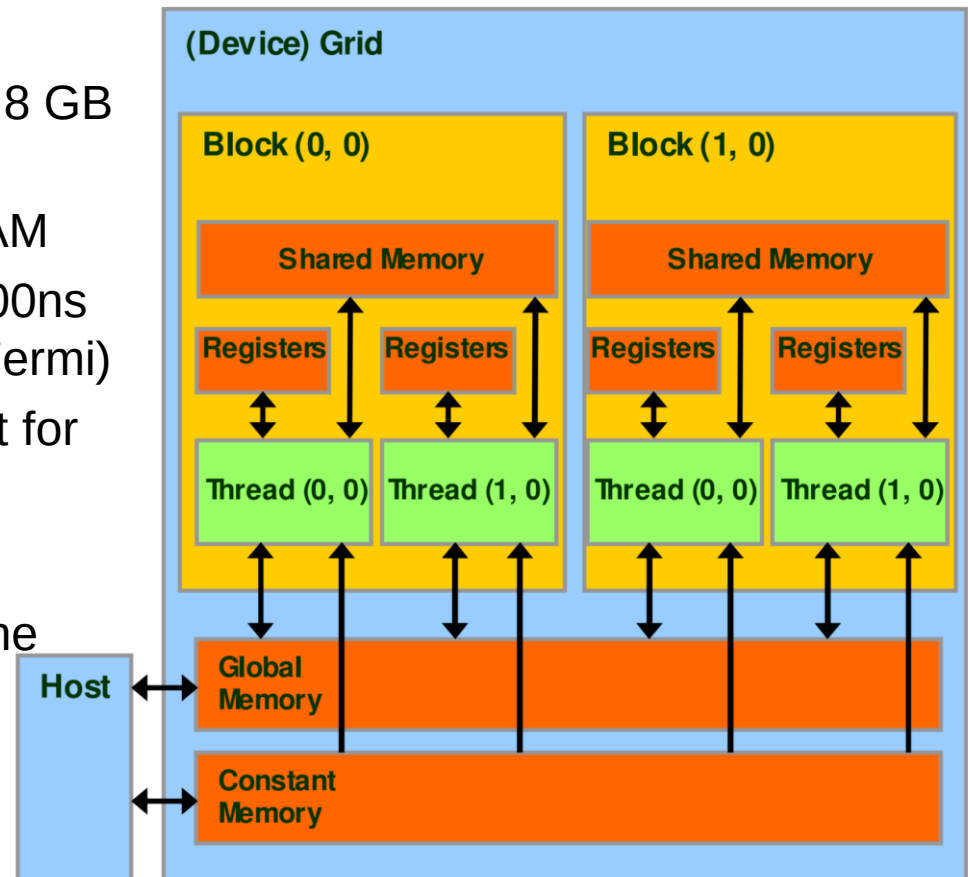


ALU: arithmetic
logic unit

An 8 units wide SIMD core: one fetch & decode
for multiple execution units and contexts

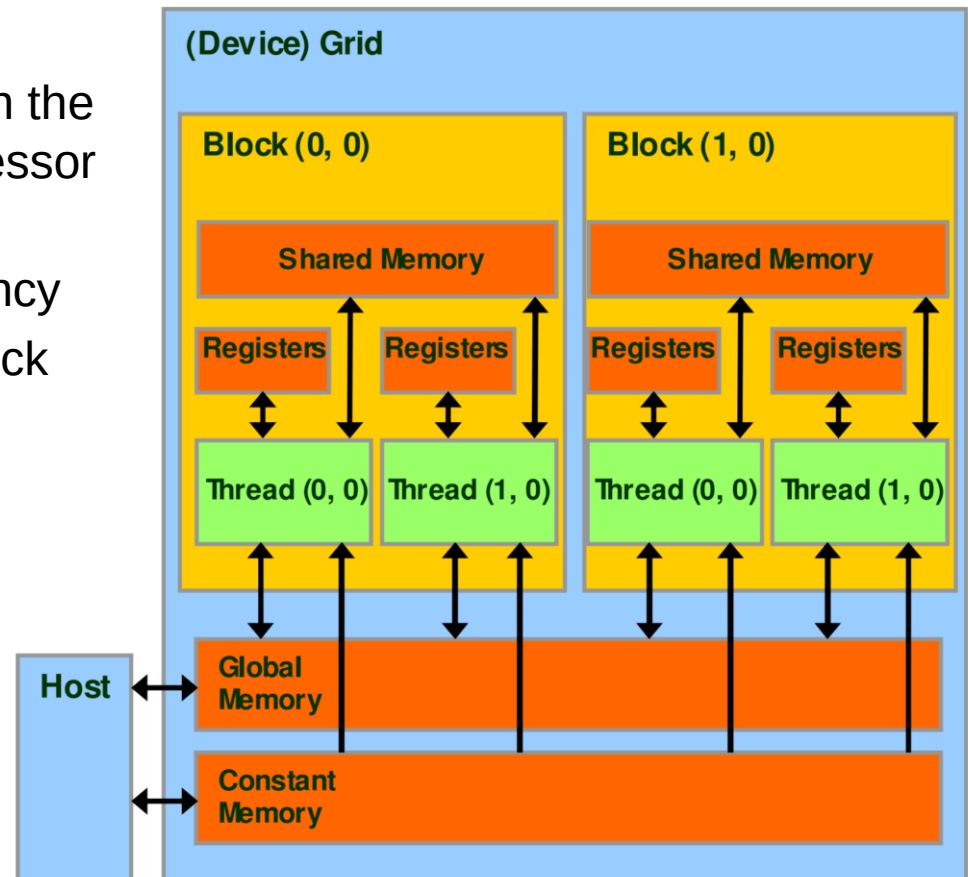
GPGPU Memory Architecture

- Global memory
 - Main memory in GPU (e.g, 8 GB for Nvidia GTX2080)
 - Located in an external DRAM
 - Relatively high latency (~300ns on Kepler and ~600ns on Fermi)
 - Slowest IO on GPU (except for accessing host memory)
- Constant memory
 - Memory with a special cache
 - Store constant values
 - Read only
 - Fast to access



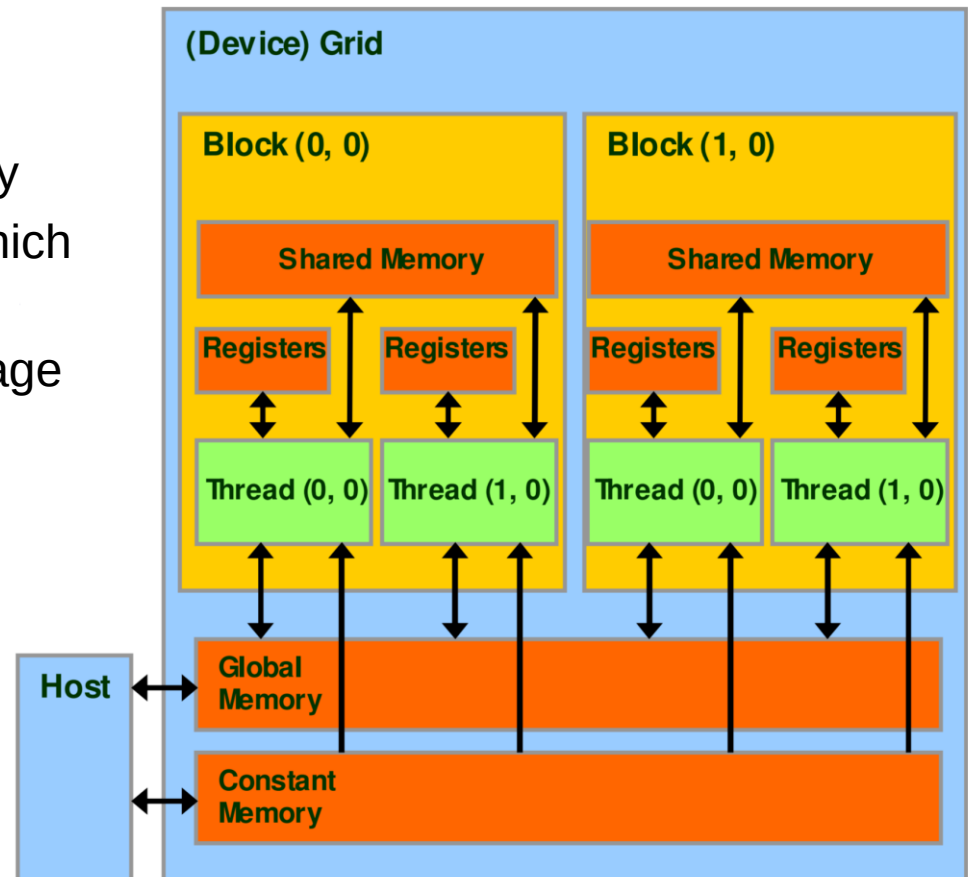
GPGPU Memory Architecture

- Shared memory
 - Very fast memory located in the single streaming multiprocessor (SM)
 - Located on-chip, ~5ns latency
 - Share memory for each block
- Registers
 - Fastest “memory”
- Local Memory
 - Everything cannot fit in the registers
 - Correspond to each thread
 - Stored in global memory

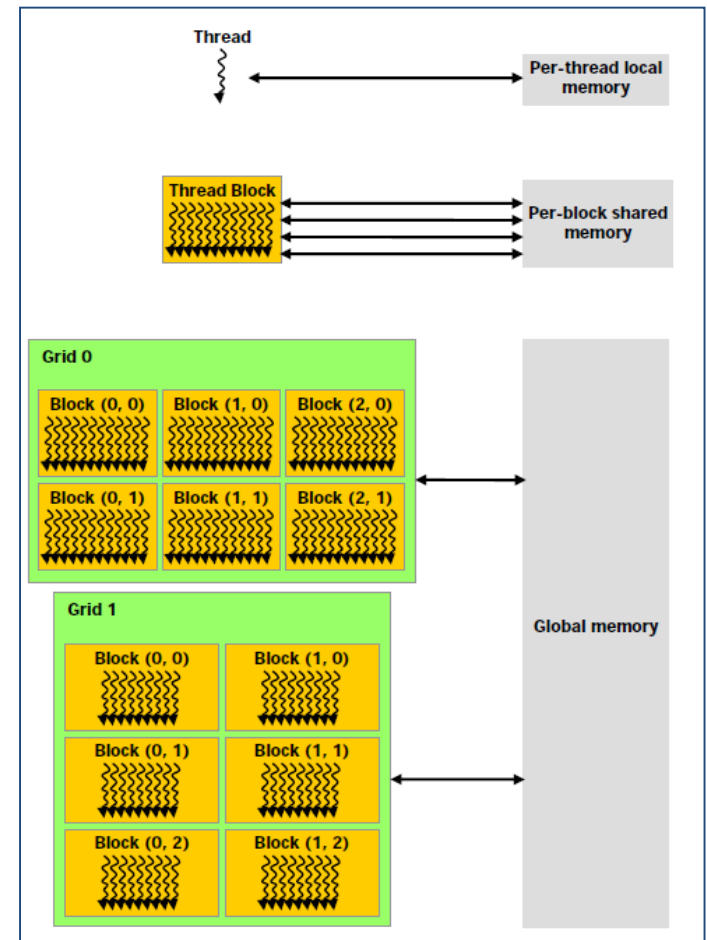
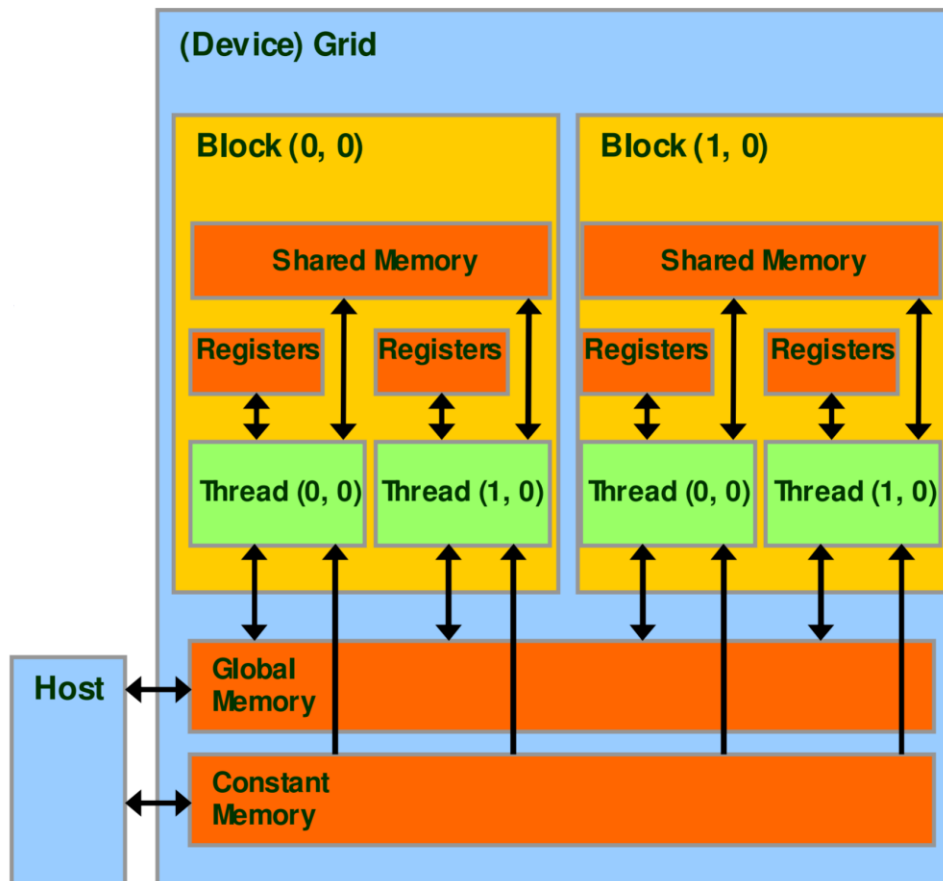


GPGPU Memory Architecture

- Texture memory
 - Read and write
 - Optimized for spatial locality
 - Very fast to read values, which need interpolations
 - 1, 2, and 3 dimensional image
 - 1, 2, and 4 channels

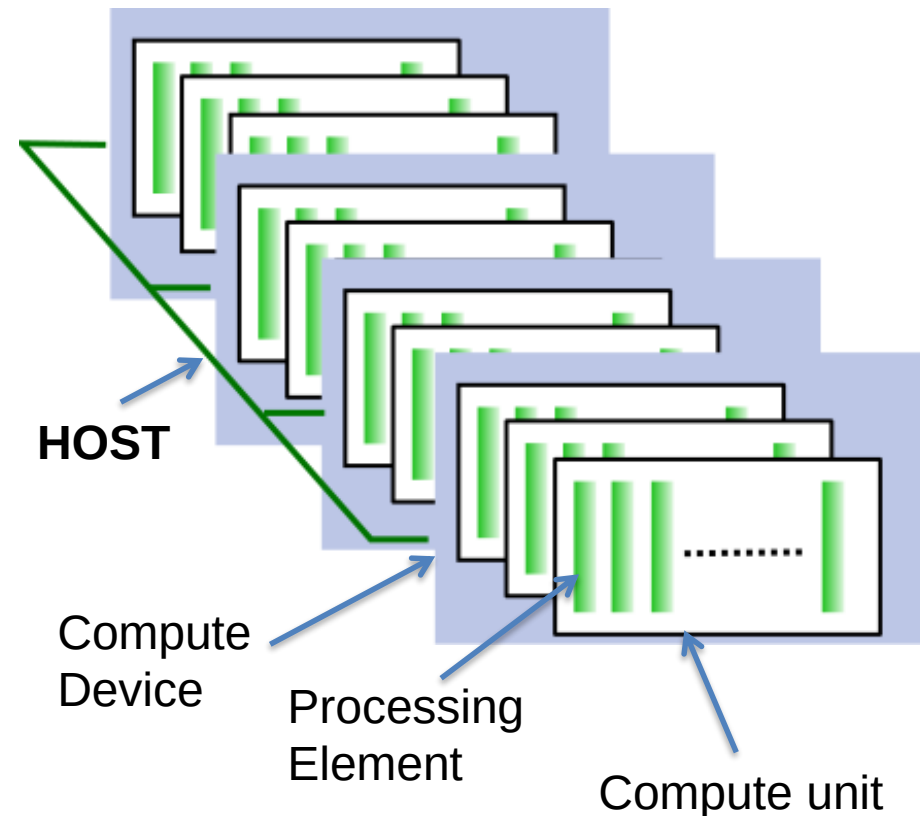


GPGPU Architecture (CUDA)



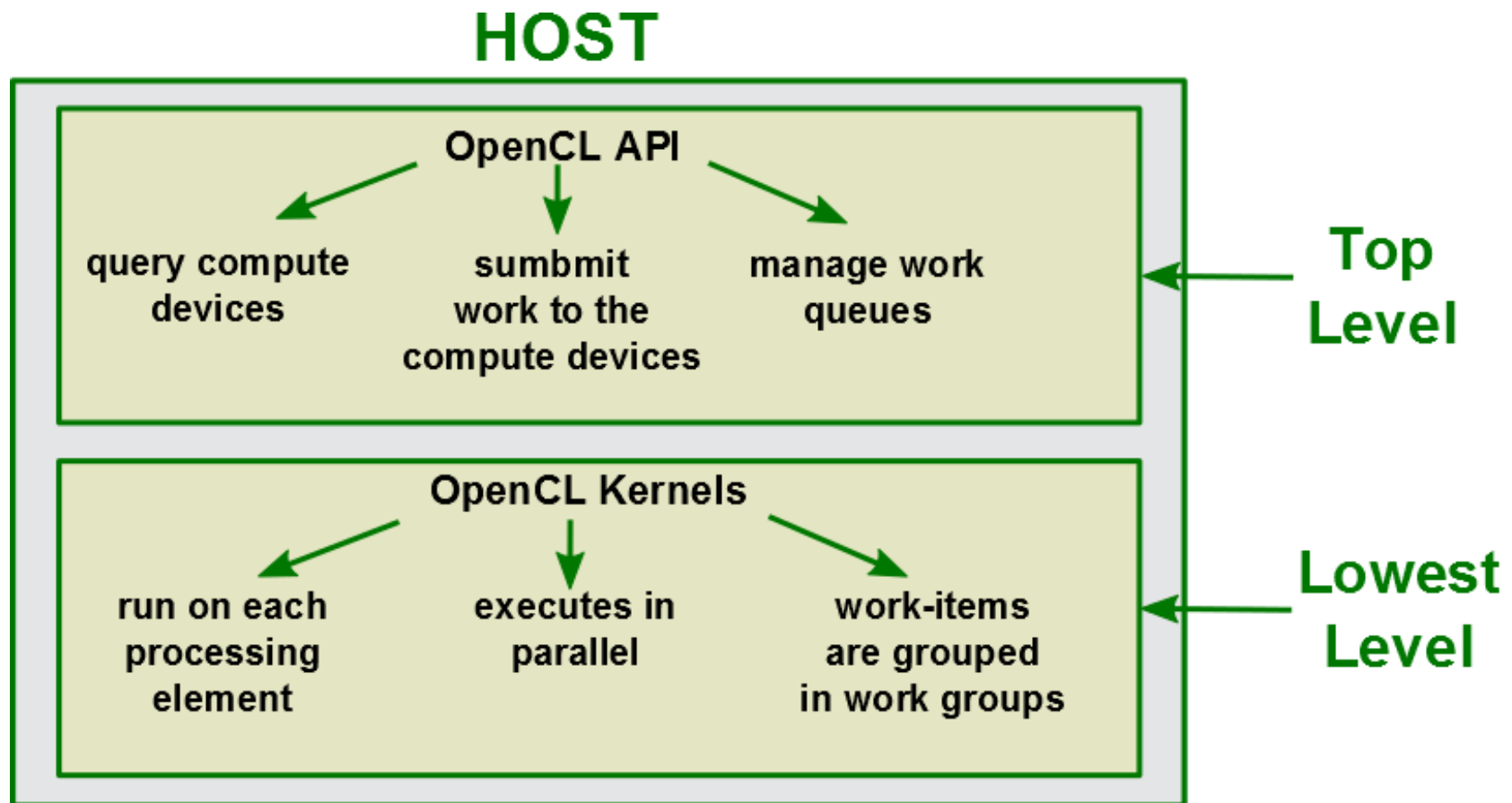
OpenCL Platform

- The OpenCL platform includes several components
 - One Host (CPU-based)
 - Compute Devices (GPU)
 - Compute units (groups of execution and arithmetic units)
 - Processing elements (executing OpenCL Kernels)
- *Serial code* executes in the Host
- *Parallel code* executes in the Compute devices



Woolley2011

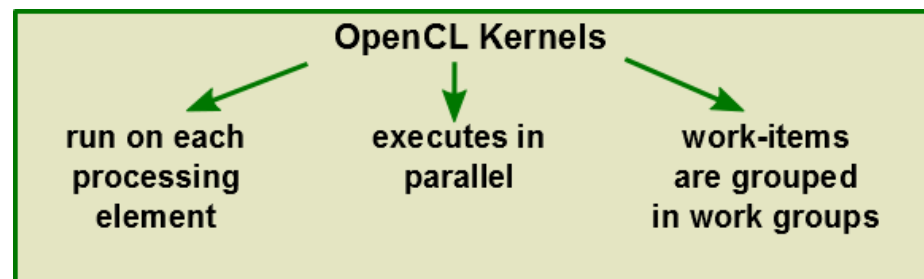
OpenCL: How the execution works



OpenCL Kernels

- An OpenCL Kernel represents the code executed by a Work item
- Kernels process in parallel: their execution needs to be independent from one another
- The code executed by the kernels needs to consider the required memory access

Lowest Level



Example of an OpenCL Kernel

```
void
kernel_value_addition(int n,
                      float *a,
                      float value)
{
    for(int i = 0; i < n; i++)
        a[i] += value
}
```

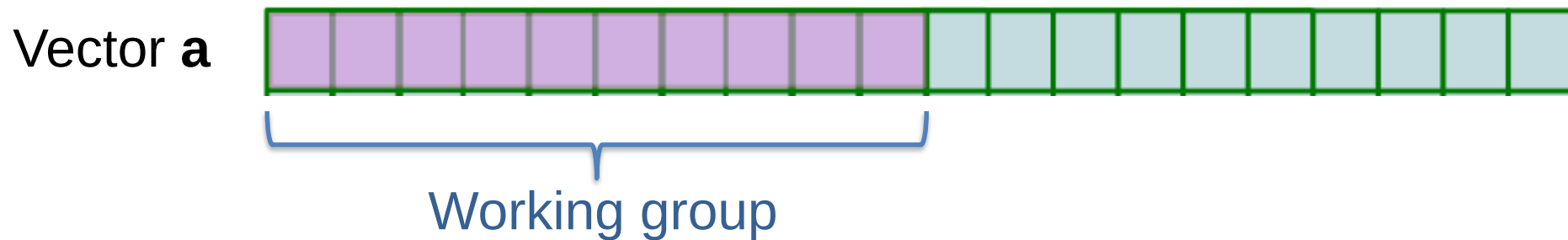
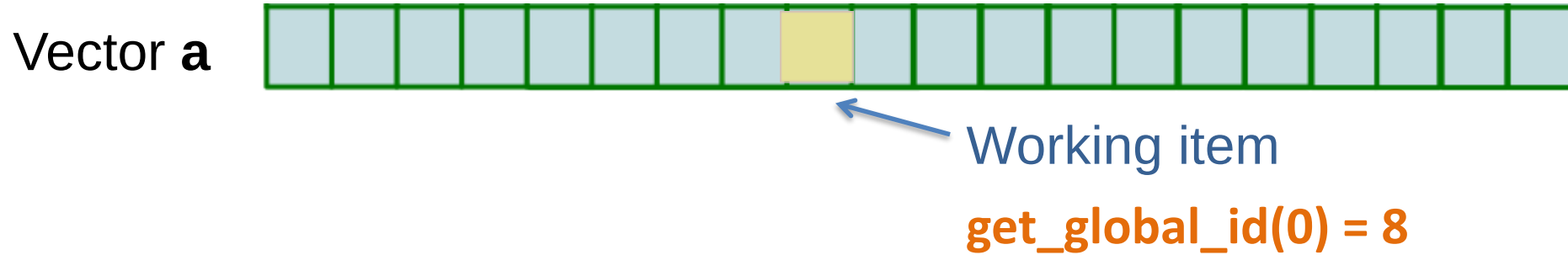
- Standard loop: The same code is repeated serially, one element after the other

```
void
kernel_value_addition(int n,
                      global float *a,
                      float value)
{
    int iGID = get_global_id(0);
    if(iGID >= n)
        return;

    a[iGID] += value;
}
```

- OpenCL kernel: The same code is executed at each thread in parallel

Work Domain 1D



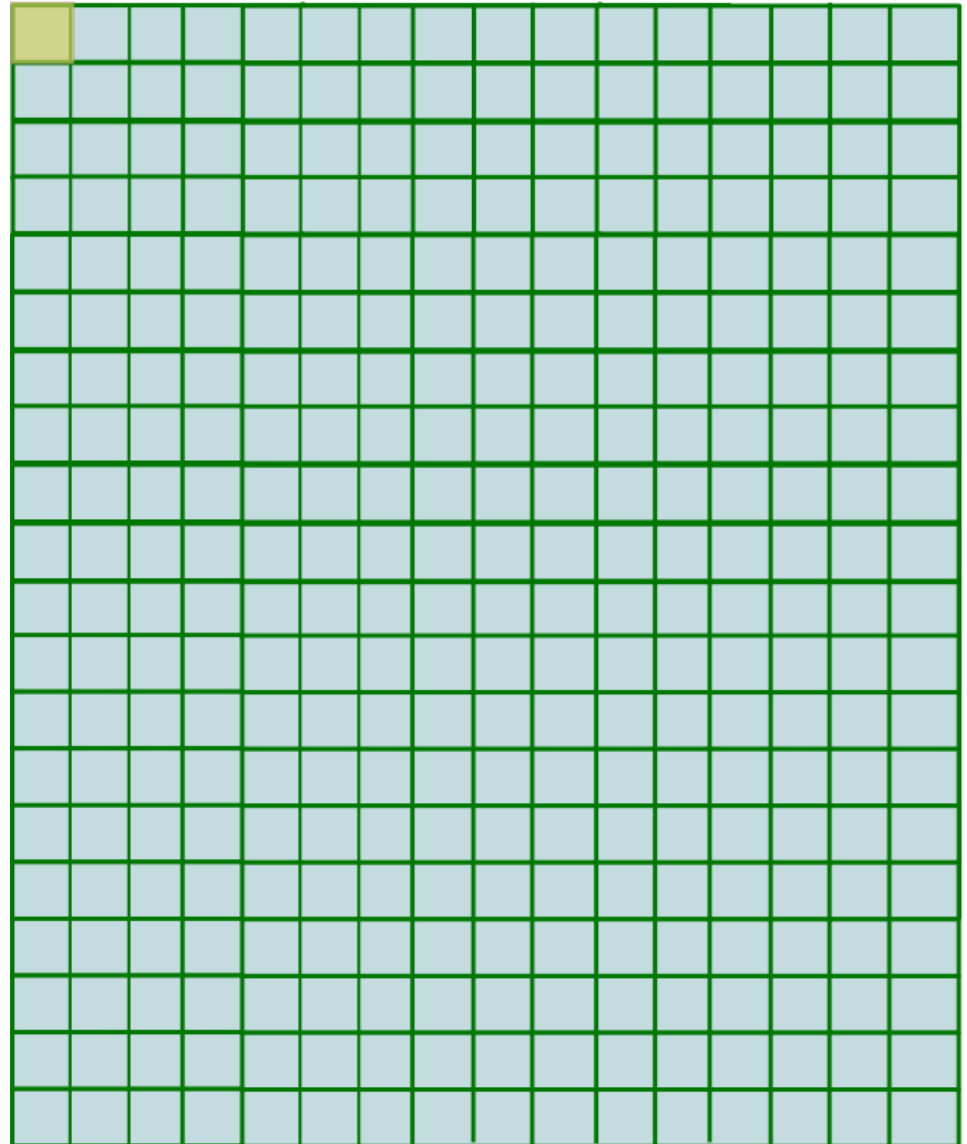
$\text{get_global_size} = 20$

$\text{get_local_size} = 10$

Work Domain in 2D

- The image represents the global dimension of the working domain (16 x 20)
- The global domain is divided into work items (e.g. a pixel)

get_global_id(0) = 0
get_global_id(1) = 0





Important OpenCL Commands and Examples

Structure of the Host Program

- The host program is the code that runs on the host to setup the environment for the OpenCL program and manages the kernels
- 1) Define a **context**, **device** and **queue**
 - Context: the environment within which kernels execute and in which synchronization and memory management is defined
 - Device: the GPU
 - Queue: all commands for the device are submitted through a queue
 - 2) Create and build the **program**
 - 3) Define **memory** objects
 - 4) Define the **kernel**
 - 5) Submit **commands** (transfer memory, execute kernel)



Example 1: Adding a value on a vector

- We split the code into host and device code
- The device code is written in a *.cl file which will be loaded later by the host code
- Each *.cl file must contain a kernel, which will be executed (i.e. entrance point of the program)
- The *.cl file is written using **Java syntax**

Example 1: Adding a value on a vector (device code)

- Example of device code:
- The kernel is called ***add*** and receives a
 - constant integer ***n***
 - An array ***a***
 - And a constant float ***value***
- Then the ***value*** is added to each element of the vector
- If the global id is greater than the vector length ***n*** nothing is done

```
kernel void add(int n,  
                global float *a,  
                float value)  
{  
    int iGID = get_global_id(0);  
    if(iGID >= n)  
        return;  
  
    a[iGID] += value ;  
}
```



Example 1: Adding a value on a vector (host code)

```
import numpy as np
import pyopencl as cl

dim = 500000
a_np = np.random.rand(dim).astype(np.float32) # create random vector

platform = cl.get_platforms() # get platforms (i.e. GPU/CPU)
GPU = platform[1].get_devices() # use the GPU which is in 2nd pos.

ctx = cl.Context(GPU) # create context (with GPU)

queue = cl.CommandQueue(ctx) # create commandqueue to communicate
# between host and device

mf = cl.mem_flags
a_g = cl.Buffer(ctx, mf.READ_WRITE | mf.COPY_HOST_PTR, hostbuf=a_np) # convert numpy array to openCL array

kernel = open("openCLVectorAdd.cl").read() # load .cl file
prg = cl.Program(ctx, kernel).build() # Build .cl file

prg.add(queue, a_np.shape, None, np.int32(dim), a_g, np.float32(20.0) ) # call kernel (queue, worksize and inputs)

res_np = np.empty_like(a_np) # create np array to write in the result
cl.enqueue_copy(queue, res_np, a_g) # copy result from device
```

Create OpenCL texture object in the host

- You can create textures using

```
inputImageCL = cl.image_from_array(ctx, array_np, num_channels=None, mode='r', norm_int=False)
```

- *array_np* is a 2d numpy array
 - *num_channels* is simply 1 in our exercise because grey scale images are used
 - *mode='r'* for *read_only* and *mode='w'*, for *write_only*.
 - If *norm_int* is *True*, then the integer values are normalized to a floating point scale of 0..1 when read.
- In the OpenCL kernel (.cl) file, the variables are specified as the following

```
read_only image2d_t img1CL
```

```
write_only image2d_t img2CL
```

OpenCL texture read and write in the kernel

- First we need to define a [sampler](#) [1]

```
sampler_t sampler = CLK_NORMALIZED_COORDS_FALSE | CLK_ADDRESS_CLAMP_TO_EDGE | CLK_FILTER_NEAREST;
```

- Define the coordinates of the pixel you want to read

```
int2 pixelCoords = (int2)(x, y); // Replace x and y with the actual pixel coordinates
```

```
float2 pixelCoords = (float2)(x + 0.5f, y + 0.5f); // Add 0.5f to the coordinates
```

- Use `read_imagef` [2] to read the pixel value

```
float4 pixelValueRGBA = read_imagef(inputImageCL, sampler, pixelCoords);
```

```
float pixelValue = pixelValueRGBA.x;
```

- Use `write_imagef` [3] to update the pixel value

```
write_imagef(resultImageCL, pos, pixelValue);
```

[1] https://man.opengl.org/sampler_t.html

[2] https://man.opengl.org/read_imagef2d.html

[3] https://man.opengl.org/write_image2d.html

OpenCL texture read and write

Note:

- when using float2 pixel coordinates with the `read_imagef` function in OpenCL, you need to add 0.5f to the coordinates to ensure correct sampling. This adjustment is necessary because image coordinates in OpenCL are specified as the center of the pixel, rather than the pixel's corner.
- when using int2 pixel coordinates with the `read_imagef` function in OpenCL, you don't need to add 0.5. Unlike float2 coordinates, int2 coordinates are treated as the corner of the pixel, so no adjustment is required.



Copy memory back from GPU to CPU

- For texture memory

```
cl.enqueue_copy(queue, res_np, r_openCL, origin=(0, 0), region= r_openCL.shape)
```

- For normal buffer

```
cl.enqueue_copy(queue, res_np, r_openCL)
```



Hints

- For `phantom_np` (numpy array) and `phantom_cl` (OpenCL texture), you can print `phantom_np.shape` and `phantom_cl.shape` to check their dimensions, since the dimension order might be changed
- Please convert all data type from 64 bit to 32 bit using `np.float32` and `np.int32` because most GPU works on single precision data as default



References

- OpenCL cheat sheet: <https://www.khronos.org/files/opencvcl-1-2-quick-reference-card.pdf>
- PyopenCL documentation: <https://documen.tician.de/pyopencl/>



Questions?